

Technical Documentation

Project: RAG-Based Customer Support Assistant

1. Introduction

Retrieval-Augmented Generation (RAG) mitigates Large Language Model (LLM) hallucinations by grounding generation in external, verified knowledge. For customer support, standard RAG is insufficient because support workflows require stateful interactions, decision-making, and human escalation. This system solves this by integrating LangGraph, shifting the architecture from a linear pipeline to an intelligent, agentic workflow capable of executing autonomous support resolutions or routing to Human-in-the-Loop (HITL) fallback.

2. System Architecture Explanation

The architecture is divided into two decoupled pipelines:

- Asynchronous Ingestion Pipeline: Converts static PDFs into normalized, mathematically searchable vector embeddings stored in ChromaDB. This runs offline or via cron jobs when documentation updates.
- Synchronous Inference Graph: A LangGraph state machine triggered by user input. It dynamically routes the query to either a Vector Store Retriever (for standard queries) or a HITL node (for complex/escalated queries), maintaining a shared GraphState payload throughout the execution cycle.

3. Design Decisions

- Chunk Size (512 tokens, 50 overlap): Chosen to balance granularity and context. 512 tokens capture complete conceptual paragraphs (preventing context fragmentation), while a 50-token overlap ensures pronouns and cross-references at the edges of chunks are not lost.
- Embedding Strategy: Utilized BAAI/bge-small-en-v1.5. It ranks exceptionally high on the MTEB leaderboard for retrieval while being small enough to run locally without incurring cloud API costs.
- Retrieval Approach (MMR): Maximal Marginal Relevance (MMR) is favored over standard Cosine Similarity. MMR penalizes redundant documents, ensuring the LLM receives diverse contextual angles rather than three identical chunks.
- Prompt Design Logic: System prompts enforce strict JSON compliance and forbid hallucination. The LLM is explicitly instructed: *"If the answer is not contained within the provided context, output a confidence score of 0.0."*

4. Workflow Explanation (LangGraph)

LangGraph orchestrates the system using a Directed Cyclic Graph (DCG) methodology.

- State: A strictly typed Python dictionary passed between nodes, accumulating context (Query → Intent → Retrieved Docs → LLM Draft).

- Nodes: Python functions that perform specific work (e.g., querying ChromaDB or invoking Groq) and mutate the State object.
- Edges: Routing functions that read the State and determine the next Node to execute.

5. Conditional Logic & Routing

Routing is executed by a lightweight LLM call optimized for low latency.

- Intent Detection: The Router Node analyzes the query. If it detects escalation keywords (e.g., "refund", "supervisor") or high frustration via sentiment analysis, it bypasses retrieval and routes to HITL.
- Evaluation Routing: Post-generation, an evaluator checks the LLM's self-assigned confidence score. If confidence < 0.8, the graph routes to HITL rather than sending a potentially inaccurate response to the user.

6. HITL Implementation

- Mechanism: LangGraph's `interrupt_before=["hitl_node"]` acts as an execution breakpoint. The graph pauses indefinitely, saving its state in memory.
- Role: An agent reviews the LLM's drafted response (if any) and the user's query via an admin UI, inputting a manual resolution.
- Benefits & Limitations: Ensures 100% safety for high-stakes queries. However, it introduces human-bound latency, requiring asynchronous UI patterns (e.g., WebSockets) so the user doesn't face a hanging HTTP request.

7. Challenges & Trade-offs

- Retrieval Accuracy vs. Speed: Using MMR adds computational overhead compared to naive top-K similarity search. *Trade-off accepted* because customer support requires precise, non-repetitive context.
- Cost vs. Performance: Groq (Llama-3.1) was selected over OpenAI (GPT-4o). While GPT-4o has superior reasoning, Groq's LPU hardware delivers sub-second latency. For a multi-step graph where the LLM is called 2-3 times per query (Routing → Generation → Evaluation), Groq prevents cascading latency delays.

8. Testing Strategy

- Component Testing: Querying the retriever in isolation to ensure expected chunks are returned for known test questions (Recall testing).
- Graph Integration Testing: Simulating a conversation with explicit keywords ("Let me speak to a human") and asserting that the `hitl_node` is reached.
- Sample Queries:
 - *Standard*: "What is the password reset policy?" (Expect: Auto-resolution).
 - *Complex*: "Your app crashed and I lost money, refund me now." (Expect: HITL routing).

- *Out of Scope*: "How do I bake a cake?" (Expect: Low confidence → Escalation/Polite Refusal).

9. Future Enhancements

- **Memory Integration**: Migrating from stateless execution to persistent threads using LangGraph's Postgres Checkpointer, allowing the bot to remember conversational history across sessions.
- **Multi-Agent Routing**: Replacing the single Router Node with a Supervisor Agent that routes to specialized sub-graphs (e.g., a "Billing Graph" vs. a "Tech Support Graph").
- **Feedback Loop**: Logging all HITL overrides to LangSmith, creating a dataset to fine-tune the routing logic and reduce future escalations.